19-10-2026

Carrera: ICI
Semestre: I
Curso: Taller de programación.
Instructor: Luis Ponce Rosales.

TAREA 1.5 U2

Funciones y Procedimientos — Identificación y Jerarquización - Arreglos

En el siguiente juego usted debe efectuar un análisis e identificar las posibles correcciones.

Objetivos de aprendizaje:

1. **Aplicar** backtracking, revisado en la clase de cátedra, a un problema de **búsqueda de caminos en este juego del ratón**
2. **Reconocer** los 4 elementos clave: estado, caso base, opciones válidas y deshacer.
3. **Depurar** código recursivo con ayuda de impresiones por consola.

Un ratón debe salir de un laberinto representado por una matriz que se muestra continuación de este texto:

Descripción de la matriz.

0 = camino libre
1 = pared
9 = salida

```
{0, 1, 0, 0, 0},
{0, 1, 0, 1, 0},
{0, 0, 0, 1, 0},
{1, 1, 0, 0, 0},
{0, 0, 0, 1, 9}
```

El ratón empieza en la esquina superior izquierda (0,0) y debe llegar a la salida (4,4). Solo puede moverse **arriba, abajo, izquierda y derecha**.

Instrucciones:

1.- Este código inicial para trabajar debe copiarlo y completarlo.

```c
#include <stdio.h>

#define FIL 5
#define COL 5

int laberinto[FIL][COL] = {
    {0, 1, 0, 0, 0},
    {0, 1, 0, 1, 0},
    {0, 0, 0, 1, 0},
    {1, 1, 0, 0, 0},
    {0, 0, 0, 1, 9}   /* 9 = salida */
};

int solucion[FIL][COL];   /* marca el camino elegido */
```

```c
void imprimirSolucion(void) {
    int i, j;
    printf("\nCamino encontrado:\n");
    for (i = 0; i < FIL; i++) {
        for (j = 0; j < COL; j++) {
            if (solucion[i][j])
                printf(" * ");
            else if (laberinto[i][j] == 1)
                printf(" # ");
            else if (laberinto[i][j] == 9)
                printf(" S ");
            else
                printf(" . ");
        }
        printf("\n");
    }
}
```

```c
int resolver(int f, int c) {
    /* 1. Caso base: si estamos fuera o es pared o ya visitado -> return 0 */
    /* 2. Caso base: si laberinto[f][c] == 9 -> return 1 (exito) */
    /* 3. Marcar solucion[f][c] = 1 */
    /* 4. Probar los 4 movimientos recursivamente */
    /* 5. Si ninguno funciona, DESHACER: solucion[f][c] = 0; return 0 */
    return 0;
}

int main(void) {
    if (resolver(0, 0))
        imprimirSolucion();
    else
        printf("No hay salida\n");
    return 0;
}
```

2.- Analice las siguientes sugerencias para completar su código.

Condicionales que ayudan determinar acciones del return de cada función.

```c
if (f < 0 || f >= FIL || c < 0 || c >= COL) return 0;
if (laberinto[f][c] == 1) return 0;
if (solucion[f][c] == 1) return 0;    /* indica que ya fue visitado */
if (laberinto[f][c] == 9) { solucion[f][c] = 1; return 1; }
```

3.- Responda la siguiente pregunta:

a) ¿cuántas celdas tiene el camino solución?

b) imprima una matriz con el camino resuelto por el ratón.

4.- Añada otra salida a la matriz del juego y calcule cuál secuencia encuentra primero.

Especificadores de formatos más frecuentes en lenguaje C.

Especificador	Tipo de dato	Ejemplo
`%d`	Entero (`int`)	`scanf("%d", &edad);`
`%f`	Flotante (`float`)	`scanf("%f", &altura);`
`%lf`	Doble (`double`)	`scanf("%lf", &pi);`
`%c`	Carácter (`char`)	`scanf(" %c", &letra);`
`%s`	Cadena (`char[]`)	`scanf("%s", nombre);` *(sin &)*
`%.2f`	Float con 2 decimales	`printf("%.2f", 3.14159);` → `3.14`